ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Relación de Problemas Tema 3 : Herencia (PDOO)

Índice

1. Ejercicio 1 **4**
1.1. Enunciado 4
1.2. Solución 4

2. Ejercicio 2 **4**
2.1. Enunciado 4
2.2. Solución 5

3. Ejercicio 3 **6**
3.1. Enunciado 6
3.2. Solución 6

4. Ejercicio 4 **7**
4.1. Enunciado 7
4.2. Solución 7

5. Ejercicio 5 **7**
5.1. Enunciado 7
5.2. Solución 7

6. Ejercicio 6 **9**
6.1. Enunciado 9
6.2. Solución 10

7. Ejercicio 7 **10**
7.1. Enunciado 10
7.2. Enunciado 10
7.3. Solución 11

8. Ejercicio 8 **11**
8.1. Enunciado 11
8.2. Solución 12

9. Ejercicio 9 **12**
9.1. Enunciado 12
9.2. Solución 13

10. Ejercicio 10 **14**

11. Ejercicio 11 **15**
11.1. Enunciado 15
11.2. Solución 16

12. Ejercicio 12 **17**
12.1. Enunciado 17
12.2. Solucion 20

1 Ejercicio 1

1.1. Enunciado

Dada la siguiente clase en Java:

```
public class Vertebrado
{
    public ArrayList<String> partesDelAbdomen();
    public void desplazarse();
    public String comunicarse(Vertebrado vertebrado);
    protected Vertebrado obtenerCopia();
}
```

Indica con una cruz en la casilla correspondiente según si la declaración de los siguientes métodos de la clase `Mamifero`, subclase de `Vertebrado`, sobrecargan (*overloading*) o redefinen (*overriding*) a los de la clase `Vertebrado`.

1.2. Solución

Método	**Sobrecarga**	**Redefinición**
public ArrayList<String>partesDelAbdomen()		X
public void desplazarse(Modo m)	X	
public String comunicarse(Vertebrado vertebrado)		X
public String comunicarse(Mamifero mamifero)	X	
public Vertebrado obtenerCopia()		X
protected Mamiero obtenerCopia()		X

Nota:Al cambiar únicamente la visibilidad o el tipo hace que sea una redefición (Últimos dos casos)

2 Ejercicio 2

2.1. Enunciado

A partir de las siguientes clases:

```
public class Persona {
    protected String nombre;
    public Persona(String nom) { this.setNombre(nom); }
    protected String getNombre() { return this.nombre; }
    public String hablar() { return "bla bla bla"; }
}
```

```
public class Estudiante extends Persona {
    public String carrera;
    public int curso;
    public Estudiante(String nom, String carr, int cur) {
        super(nom);
```

```
        carrera = carr;
        curso = cur;
    }
    public void estudiar() { System.out.println("estudiando"); }
}
```

Implementa la clase `EstudianteInformatica` que hereda de `Estudiante`. Debe tener:

- Una nueva variable de instancia que es una colección de `String` con los dispositivos que utiliza (por ejemplo, `PC`, `tablet`, `smartphone`).
- Métodos para consultar esa variable.
- Una redefinición del método `estudiar()` para que además de estudiar como los otros alumnos, diga que estudia con el último dispositivo de su colección.
- Un constructor para inicializar la clase.

Implementa este problema en Java y Ruby:

2.2. Solución

```
    public class EstudianteInformatica extends Estudiante {
        private ArrayList<String> dispositivos;
        public EstudianteInformatica(string nombre, string curso){
            super(nombre, "Informatica", curso);
            dispositivos = new ArrayList<String>();
        }

        @Override
        public void Estudiar(){
            System.out.println("Estudiando Informatica con " + dispositivos
                .get(dispositivos.size() - 1));
        }
    } // Fin de la clase EstudianteInformatica
```

Listing 1: Clase Estudiante de Informática

```
    def EstudianteInformatica < Estudiante
        def initialize (nombre, curso)
            super(nombre, "Informatica", curso)
            @dispositivos = [] # @dispositivos = Array.new
        end

        def Estudiar
            super.Estudiar
            puts "Estudiando Informatica con #{@dispositivos[-1]}"
        end
    end
```

Listing 2: Clase Estudiante de Informática

3 Ejercicio 3

3.1. Enunciado

Nota: En las diapositivas de la Relación hay un error, donde pone 3, no es el ejercicio 3 es la parte 2 del ejercicio 2, es decir, otra clase, el ejercicio 3 es el marcado con el número 4.

Dada la siguiente clase abstracta en Java:

```java
abstract class Transporte {
    protected String marca;

    protected String getMarca() {
        return this.marca;
    }

    protected void setMarca(String marc) {
        this.marca = marc;
    }

    public abstract String hacerRuta(String origen, String destino);
}
```

Crea dos nuevas clases, `Bicicleta` y `NaveEspacial`, que hereden de `Transporte` e implementen el método `hacerRuta` de forma diferente:

3.2. Solución

```java
    public class Bicicleta extends Transporte {
        @Override
        public String hacerRuta(String origen, String destino) {
            return "Pedaleando desde " + origen + " hasta " + destino;
        }
    }

    def NaveEspacial < Transporte
        def hacerRuta(origen, destino)
            "Volando desde #{origen} hasta #{destino} por el espacio"
        end
    end
```

Listing 3: Clase Bicicleta

Diagrama UML

4 Ejercicio 4

4.1. Enunciado

Hacer el ejercicio anterior, pero en Ruby.

4.2. Solución

```ruby
    def Bicicleta < Transporte
        def hacerRuta(origen, destino)
            "Pedaleando desde #{origen} hasta #{destino}"
        end
    end

    def NaveEspacial < Transporte
        def hacerRuta(origen, destino)
            "Volando desde #{origen} hasta #{destino} por el espacio"
        end
    end
```

Listing 4: Clase Bicicleta

5 Ejercicio 5

5.1. Enunciado

Implementa en Java una clase paramétrica a partir de la cual se puedan definir clases de grupos de personas con un líder, como por ejemplo grupos de música con un cantante, o empresas con un jefe. Implementa también alguna de esas clases a partir de la clase paramétrica definida.

5.2. Solución

A continuación, se implementa una clase paramétrica en Java para definir grupos de personas con un líder. Posteriormente, se muestra un ejemplo de uso creando un grupo de música con un cantante.

```java
// Clase paramétrica genérica
public class Grupo<T> {
    private T lider;
    private List<T> miembros;

    public Grupo(T lider) {
        this.lider = lider;
        this.miembros = new ArrayList<>();
    }

    public T getLider() {
        return lider;
    }
```

```java

    public void setLider(T lider) {
        this.lider = lider;
    }

    public void agregarMiembro(T miembro) {
        this.miembros.add(miembro);
    }

    public List<T> getMiembros() {
        return miembros;
    }

    @Override
    public String toString() {
        return "Líder: " + lider + ", Miembros: " + miembros;
    }
}

// Ejemplo de uso con un grupo de música
public class Musico {
    private String nombre;

    public Musico(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return nombre;
    }
}

public class Main {
    public static void main(String[] args) {
        Musico cantante = new Musico("Freddie Mercury");
        Grupo<Musico> banda = new Grupo<>(cantante);

        banda.agregarMiembro(new Musico("Brian May"));
        banda.agregarMiembro(new Musico("Roger Taylor"));
        banda.agregarMiembro(new Musico("John Deacon"));

        System.out.println(banda);
    }
}
```

Si queremos implementar esta misma solución en Ruby, sería como sigue:

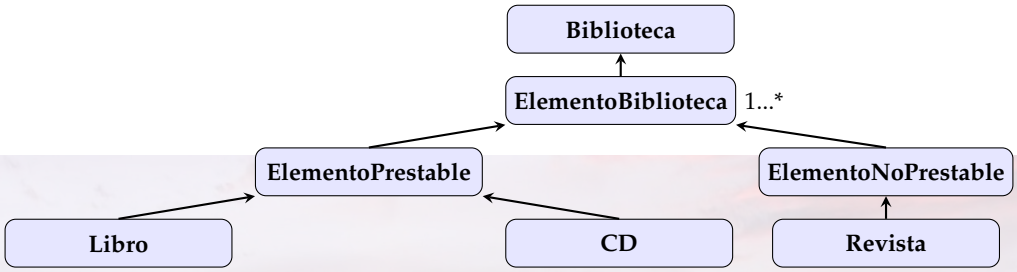
```ruby
# Clase paramétrica genérica
class Grupo
  attr_accessor :lider, :miembros

  def initialize(lider)
    @lider = lider
    @miembros = []
  end
```

```ruby

  def agregar_miembro(miembro)
    @miembros << miembro
  end

  def to_s
    "Líder: #{@lider}, Miembros: #{@miembros.join(', ')}"
  end
end

# Ejemplo de uso con un grupo de música
class Musico
  attr_accessor :nombre

  def initialize(nombre)
    @nombre = nombre
  end

  def to_s
    @nombre
  end
end

# Crear grupo de música
cantante = Musico.new("Freddie Mercury")
banda = Grupo.new(cantante)

banda.agregar_miembro(Musico.new("Brian May"))
banda.agregar_miembro(Musico.new("Roger Taylor"))
banda.agregar_miembro(Musico.new("John Deacon"))

puts banda
```

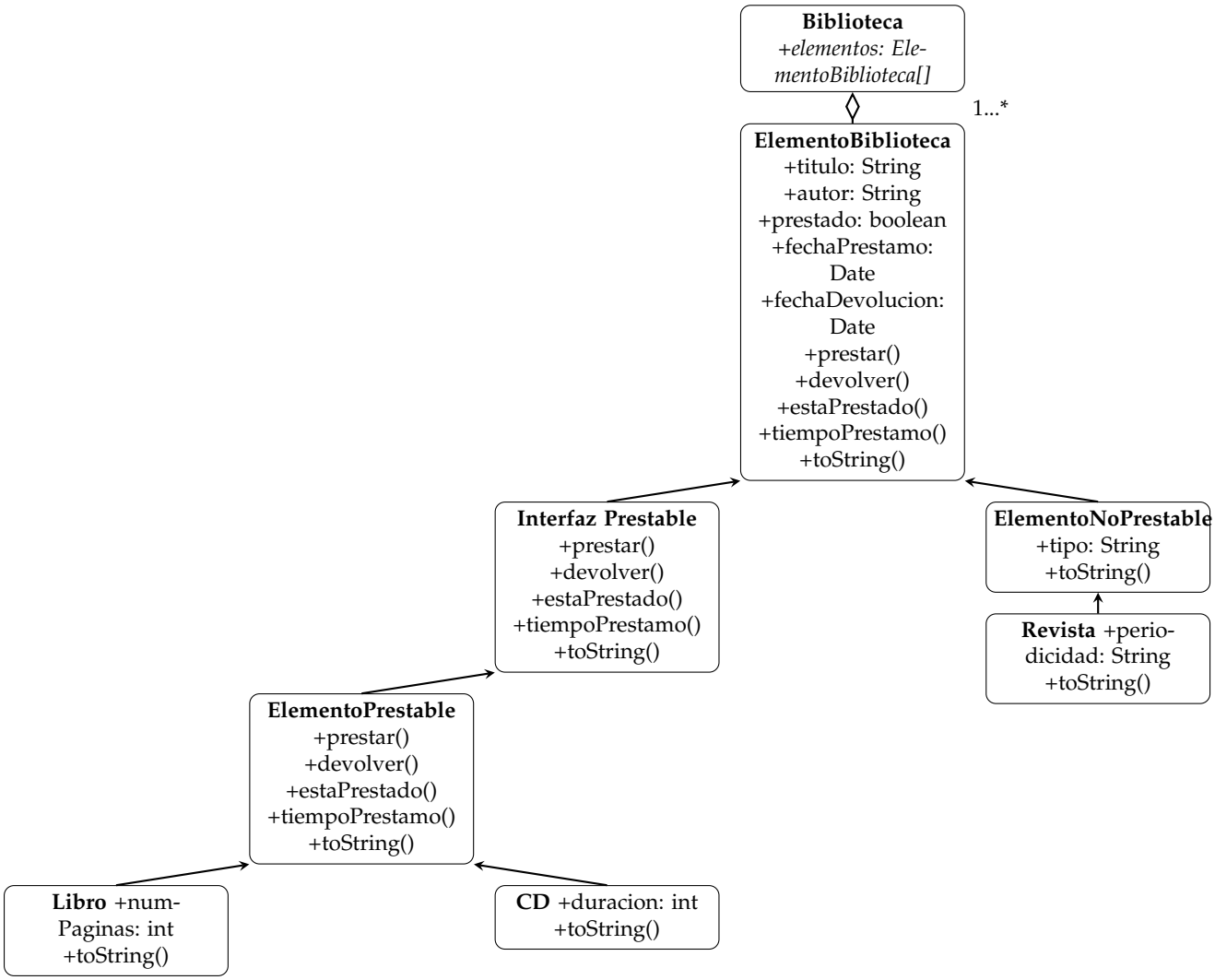
6 Ejercicio 6

6.1. Enunciado

Partiendo del siguiente diagrama de clases, modifícalo añadiendo una interfaz Prestable implementada por la clase ElementoPrestable y sus subclases. Añade a este diagrama de clases los atributos y operaciones de las distintas clases y de la interfaz Prestable.



6.2. Solución



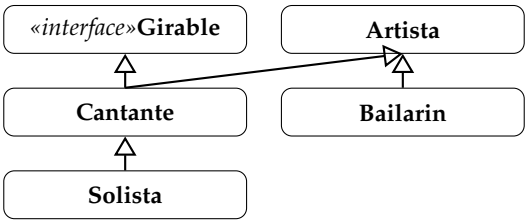
7 Ejercicio 7

7.1. Enunciado

Teniendo el cuenta el siguiente diagrama de clases, señala en la segunda columna de la tabla aquellas líneas de código que contienen un error de compilación por incompatibilidad de tipos (ECIT) y escribe en la tercera columna la corrección necesaria para evitarlos.

7.2. Enunciado

Teniendo el cuenta el siguiente diagrama de clases, señala en la segunda columna de la tabla aquellas líneas de código que contienen un error de compilación por incompatibilidad de tipos (ECIT) y escribe en la tercera columna la corrección necesaria para evitarlos.



Código	ECIT	Corrección
Girable arti=new Artista();		
Cantante cant1=new Cantante();		
Cantante sol=new Solista();		
Solista cant2=new Cantante();		
Bailarin bail= new Artista();		

Figura 1: Tabla de código y corrección

7.3. Solución

Código	ECIT	Corrección
Girable arti=new Artista();	X	Girable arti = new Cantante()
Cantante cant1=new Cantante();	✓	
Cantante sol=new Solista();	✓	
Solista cant2=new Cantante();	X	Cantante cant2=new Cantante();
Bailarin bail= new Artista();	X	Bailarin bail= new Bailarin();

Figura 2: Tabla de código y corrección

8 Ejercicio 8

8.1. Enunciado

Teniendo en cuenta:

- El diagrama de clases del ejercicio anterior
- Que se han resuelto todos los errores del ejercicio
- Que en Java la primera posición de los contenedores es la 0
- Que todos los artistas actúan, pero sólo los cantantes cantan y sólo los solistas cantan solos

Marca las líneas donde se produce un error, indicando en la segunda columna si es de compilación (C) o de ejecución (E) y en la tercera el código correcto.

Código	Error (C/E)	Corrección
List<Artista>lista = new ArrayList();		
lista.add(arti);		
lista.add(cant1);		
lista.add(sol);		
lista.add(cant2);		
lista.add(bail);		
lista.get(1).canta();		
lista.get(0).actua();		
(Solista) lista.get(3).cantaSolo();		

Figura 3: Tabla de código y corrección

8.2. Solución

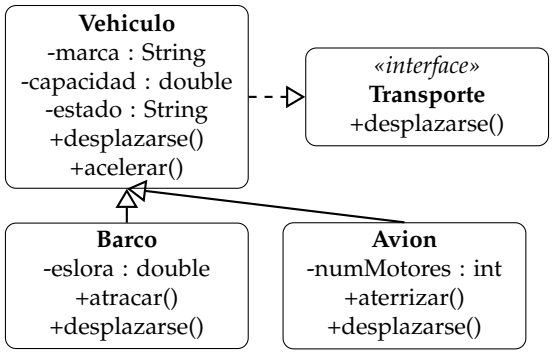
Código	Error (C/E)	Corrección
List<Artista>lista = new ArrayList();	C	List<Artista>lista = new ArrayList<Artista>();
lista.add(arti);	No	Debido a que es un cantante y es una subclase de artista
lista.add(cant1);	No	Debido a que cantante esta por debajo en el diagrama de clases
lista.add(sol);	No	Debido a que es una subclase de Cantante y esta también lo es de Artista
lista.add(cant2);	No	”
lista.add(bail);	No	”
lista.get(1).canta();	No	Es un cantante
lista.get(0).actua();	E	Es un cantante
(Solista) lista.get(3).cantaSolo();	E	en la posición 3 es un cantante

Figura 4: Tabla de código y corrección

9 Ejercicio 9

9.1. Enunciado

Dado el siguiente diagrama de UML, impleméntalo en Java. Presta atención a que el nombre de la clase Vehículo y su método desplazarse() aparecen en cursiva.



9.2. Solución

```
class Vehículo implements Transporte {
    private String marca;
    private double capacidad;
    private String estado;

    public void desplazarse() {
        System.out.println("Desplazándose");
    }

    public void acelerar() {
        System.out.println("Acelerando");
    }
}
```

Listing 5: Clase Vehiculo

```
class Barco extends Vehículo {
    private double eslora;

    public void atracar() {
        System.out.println("Atracando");
    }

    @Override
    public void desplazarse() {
        System.out.println("Desplazándose por el agua");
    }
}
```

Listing 6: Clase Barco

```
class Avion extends Vehículo {
    private int numMotores;

    public void aterrizar() {
        System.out.println("Aterrizando");
    }

```

```java
    @Override
    public void desplazarse() {
        System.out.println("Desplazándose por el aire");
    }
}
```

Listing 7: Clase Avion

```java

interface Transporte {
    void desplazarse();
}
```

Listing 8: Interfaz Transporte

10 Ejercicio 10

Teniendo presente el diagrama anterior, indica cuáles de los siguientes trozos de código son correctos y cuáles darían error de compilación o de ejecución. En caso de que den error, indica por qué y cómo lo solucionarías.

Código	Error si procede y corrección
Transporte x = new Barco(); x.atracar();	Error de compilación, ya que el tipo de la variable x es Transporte y no tiene el método atracar.
Avion av = new Avion(); av.acelerar();	Correcto.
Vehiculo v = new Vehiculo(); v.desplazarse();	Correcto.
Vehiculo v2 = new Vehiculo(); v2.acelerar();	Correcto.
Transporte t = new Avion(): Barco b= new Barco(); t = b;	Error de compilación, ya que no se puede asignar un objeto de tipo Barco a una variable de tipo Transporte.
Avion a = new Avion(); String est = a.estado;	Error de compilación, ya que el atributo estado es privado en la clase Vehiculo.
Vehiculo v3 = new Barco(); ((Barco) v3).atracar();	Correcto.
List<Transporte>listaTransportes = new ArrayList(); listaTransportes.add(new Barco()); listaTransportes.add(new Avion()); listaTransportes.add(new Barco()); for(Transporte tr: listaTransportes) tr.acelerar(); tr.atracar();	Error de compilación, ya que el método atracar() no está definido en la interfaz Transporte.
List<Transporte>otraLista = new ArrayList(); otraLista.add(new Barco()); otraLista.add(new Avion()); otraLista.add(new Barco()); for(Transporte tr: otraLista) ((Barco) tr).acelerar(); ((Barco) tr).atracar();	Error de ejecución, ya que no se puede hacer un cast de un objeto de tipo Avion a Barco.

11 Ejercicio 11

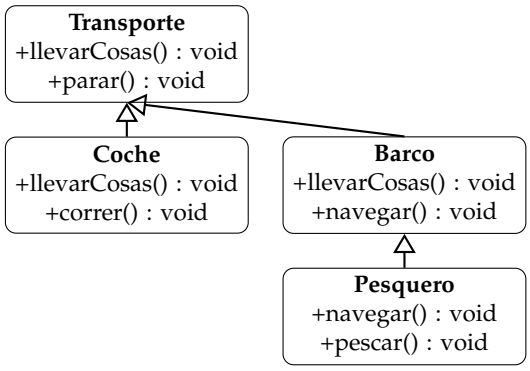
11.1. Enunciado

A partir del siguiente diagrama de clases y del código proporcionado, detecta errores de compilación y de ejecución. Corrígelos para que funcione correctamente.

```
class PruebaLigadura {
    public static void main ( String args[]){
        Coche c1 = new Coche();
        c1.llevarCosas();
```

```
            c1.correr();
            Barco b = new Barco();
            b.llevarCosas();
            b.navegar();
            b.correr();
            b=c1;
            Pesquero p = new Pesquero();
            p.navegar();
            p.pescar();
            p.llevarCosas();
            p=b;
            b=p;
            b.navegar();
            b.pescar();
            ArrayList v= new ArrayList();
            v.add(c1);
            v.add(p);
            (v.get(1)).navegar();
            (v.get(1)).llevarCosas();
        }
    }
```

Listing 9: Ejercicio del Enunciado



11.2. Solución

```
class PruebaLigadura {
    public static void main ( String args[]){
        Coche c1 = new Coche();
        c1.llevarCosas();
        c1.correr();
        Barco b = new Barco();
        b.llevarCosas();
        b.navegar();
        b.correr(); // Error: Barco no tiene el método correr
        b=c1; // Error: No se puede asignar un Coche a un Barco
        Pesquero p = new Pesquero();
        p.navegar();
        p.pescar();
        p.llevarCosas();
        p=b; // Error: No se puede asignar un Barco a un Pesquero
        b=p;
        b.navegar();
```

```
        b.pescar(); // Error: Barco no tiene el método pescar
        ArrayList<Transporte> v= new ArrayList<>();
        v.add(c1);
        v.add(p);
        ((Barco) v.get(1)).navegar();
        v.get(1).llevarCosas();
    }
}

//-----------Implementación de las clases-----------------

class Transporte {
    public void llevarCosas() {
        System.out.println("Llevando cosas");
    }
    public void parar() {
        System.out.println("Parando");
    }
}

class Coche extends Transporte {
    public void correr() {
        System.out.println("Corriendo");
    }
}

class Barco extends Transporte {
    public void navegar() {
        System.out.println("Navegando");
    }
}

class Pesquero extends Barco {
    public void pescar() {
        System.out.println("Pescando");
    }
}
```

Listing 10: Ejercicio del Enunciado

12 Ejercicio 12

12.1. Enunciado

A partir de los siguientes diagramas de clases, resuelve las cuestiones que se plantea (siempre que sean de codificación hazlo en Java y en Ruby):

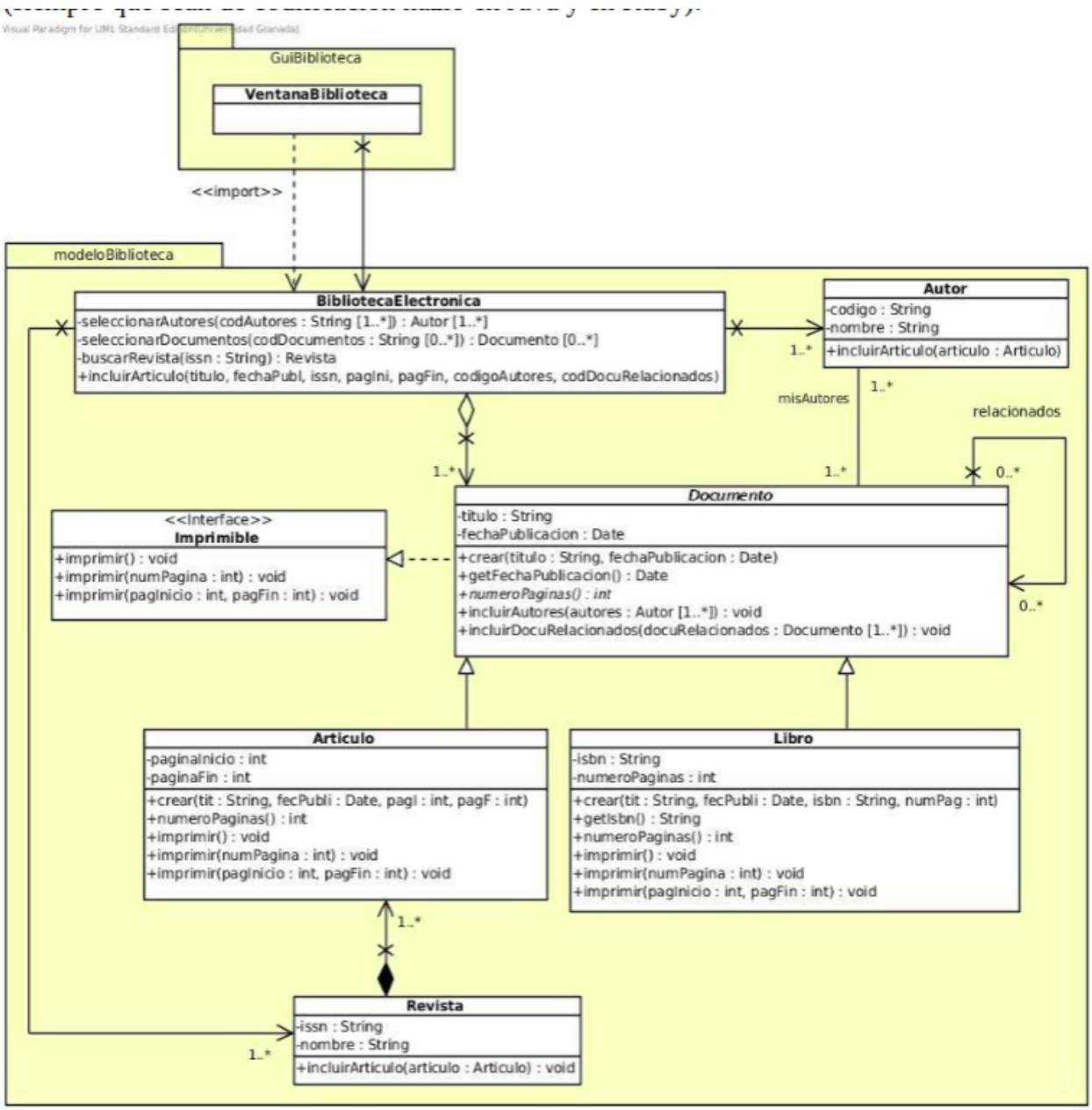


Figura 5: Diagrama de clases

- Define la clase Documento (cabecera, atributos y cabeceras de los métodos).
- Define la clase Articulo (cabecera, atributos y cabeceras de los métodos).
- Implementa el constructor que se indica en la clase Artículo.
- Define la interfaz Imprimible.
- Indica los atributos que definen el estado de la clase BibliotecaElectronica.

- La clase Documento figura en cursiva, lo que indica que es abstracta. Indica los dos motivos por los que lo es.
- En este modelo, ¿puede haber artículos que no estén ligados a una revista?
- Indica sobre las relaciones del diagrama de clases con cuál de los siguientes tipos se corresponden: asociación (AS), agregación (AG), composición (C), realización (R), herencia (H), dependencia (D).
- Escribe el contenido del fichero VentanaBiblioteca completo (pero sin añadir nada que no aparezca en el diagrama).
- En la clase Articulo, indica cuáles de sus métodos están sobrecargados o redefinidos. Justifica tu respuesta.
- Corrige el código:

```
    Imprimible docu = new Documento("Intemperie", fecha); //
        suponiendo que fecha está inicializada
    docu.imprimir();
```

- Corrige el código:

```
    Documento docu = new Libro("ISBN10102030");
    String codigo = docu.getIsbn();
```

- Rellena la siguiente tabla indicando el tipo estático y dinámico de la variable docu en las siguientes líneas de código:

Código	**Tipo estático**	**Tipo dinámico**
Documento docu = new Articulo(titulo, fecha, pagIni, pagFin);		
docu.imprimir(pagIni, pagFin);		
docu = new Libro(titulo, fecha, isbn, pags);		
docu.imprimir();		

- Indica si hay errores de compilación o ejecución en el código anterior (suponiendo que las variables titulo, fecha, pagIni, pagFin, isbn y pags han sido declaradas e inicializadas convenientemente con anterioridad). Justifica tu respuesta.
- Rellena la siguiente tabla marcando con una "X" la situación que corresponda a cada una de las líneas del siguiente bloque de código (suponiendo que las variables titulo, fecha, pagIni y pagFin han sido declaradas e inicializadas convenientemente con anterioridad).

Código	Ningún error	Sólo error de compilación	Sólo error de ejecución
Imprimible imp = new Articulo(titulo, fecha, pagIni, pagFin);			
Documento docu = imp;			
imp.numeroPaginas();			
((Libro) docu).getIsbn();			

12.2. Solucion

Define la clase Documento (cabecera, atributos y cabeceras de los métodos).

```
public abstract class Documento {
    private String titulo;
    private Date fecha;

    public String getTitulo();
    public Date getFecha();
    public void imprimir();
}
```

Define la clase Articulo (cabecera, atributos y cabeceras de los métodos).

```
public class Articulo extends Documento {
    private int pagIni;
    private int pagFin;

    public Articulo(String titulo, Date fecha, int pagIni, int pagFin);
    public int getPagIni();
    public int getPagFin();
    @Override
    public void imprimir();
}
```

Implementa el constructor que se indica en la clase Artículo.

```
public Articulo(String titulo, Date fecha, int pagIni, int pagFin) {
    super(titulo, fecha);
    this.pagIni = pagIni;
    this.pagFin = pagFin;
}
```

Define la interfaz Imprimible.

```
public interface Imprimible {
    void imprimir();
}
```

Indica los atributos que definen el estado de la clase BibliotecaElectronica.

```java
public class BibliotecaElectronica {
    private List<Documento> documentos;
}
```

La clase Documento figura en cursiva, lo que indica que es abstracta. Indica los dos motivos por los que lo es.

- No se puede instanciar directamente.
- Contiene al menos un método abstracto.

En este modelo, ¿puede haber artículos que no estén ligados a una revista?

- No, todos los artículos deben estar ligados a una revista.

Indica sobre las relaciones del diagrama de clases con cuál de los siguientes tipos se corresponden: asociación (AS), agregación (AG), composición (C), realización (R), herencia (H), dependencia (D).

- Documento - Articulo: H
- Documento - Libro: H
- Documento - Revista: H
- BibliotecaElectronica - Documento: AG
- Articulo - Revista: C
- Documento - Imprimible: R

Escribe el contenido del fichero VentanaBiblioteca completo (pero sin añadir nada que no aparezca en el diagrama).

```java
public class VentanaBiblioteca {
    private BibliotecaElectronica biblioteca;

    public VentanaBiblioteca(BibliotecaElectronica biblioteca) {
        this.biblioteca = biblioteca;
    }

    public void mostrar() {
        // Implementación del método mostrar
    }
}
```

En la clase Articulo, indica cuáles de sus métodos están sobrecargados o redefinidos. Justifica tu respuesta.

- El método `imprimir()` está redefinido porque sobrescribe el método de la clase `Documento`.

Corrige el código:

```
Imprimible docu = new Articulo("Intemperie", fecha, 1, 10);
docu.imprimir();
```

Corrige el código:

```
Libro docu = new Libro("ISBN10102030");
String codigo = docu.getIsbn();
```

Rellena la siguiente tabla indicando el tipo estático y dinámico de la variable docu en las siguientes líneas de código:

Código	**Tipo estático**	**Tipo dinámico**
Documento docu = new Articulo(titulo, fecha, pagIni, pagFin);	Documento	Articulo
docu.imprimir(pagIni, pagFin);	Documento	Articulo
docu = new Libro(titulo, fecha, isbn, pags);	Documento	Libro
docu.imprimir();	Documento	Libro

Indica si hay errores de compilación o ejecución en el código anterior (suponiendo que las variables titulo, fecha, pagIni, pagFin, isbn y pags han sido declaradas e inicializadas convenientemente con anterioridad). Justifica tu respuesta.

- Hay un error de compilación en `docu.imprimir(pagIni, pagFin);` porque el método `imprimir()` no acepta parámetros en la clase `Documento`.

Rellena la siguiente tabla marcando con una "X" la situación que corresponda a cada una de las líneas del siguiente bloque de código (suponiendo que las variables titulo, fecha, pagIni y pagFin han sido declaradas e inicializadas convenientemente con anterioridad).

Código	**Ningún error**	**Sólo error de compilación**	**Sólo error de ejecución**
Imprimible imp = new Articulo(titulo, fecha, pagIni, pagFin);	X		
Documento docu = imp;		X	
imp.numeroPaginas();		X	
((Libro) docu).getIsbn();			X